

LINT REPORT

Índice

1. Resumen
2. Introducción
3. Contenidos: Análisis de Code Smells
4. Conclusiones
5. Bibliografía

1. Resumen

El presente informe recoge el resultado del análisis estático realizado con SonarLint sobre el proyecto en Eclipse. La herramienta reportó un total de 142 avisos distribuidos en 100 archivos fuente. Tras revisar cada aviso, se identificaron 11 tipos de code smell distintos que afectan a distintos componentes del sistema.

El aviso más frecuente (java:S00120) afecta a aproximadamente 75 archivos y está relacionado con la convención de nombres de paquetes. Este aviso se considera inocuo en su totalidad ya que la nomenclatura responde a las convenciones propias del framework y modificarla implicaría una refactorización masiva sin beneficio funcional. El resto de avisos (java:S1948, java:S1125, java:S2589, java:S1117, java:S1068, java:S1128, java:S1134, java:S1481, java:S1854) afectan a ficheros concretos y se detallan individualmente en la sección 4, junto con la justificación de su inocuidad. Todos los avisos incluidos en este informe han sido analizados y se ha concluido que no representan riesgos para la corrección, seguridad ni mantenibilidad del sistema.

2. Introducción

Este informe presenta los resultados del análisis de calidad de código estático realizado con la herramienta SonarLint, integrada en el entorno de desarrollo Eclipse, sobre el proyecto desarrollado por el grupo de trabajo. El objetivo del análisis es identificar posibles defectos de código (code smells) que puedan comprometer la calidad, mantenibilidad o seguridad del software.

SonarLint reportó un total de 142 avisos en 100 recursos distintos. Conforme a las instrucciones del profesor, se han ignorado todos los avisos generados en el Acme Framework, ya que este componente no forma parte del código desarrollado por el equipo. Para cada aviso restante, el equipo ha evaluado si corresponde a un defecto real corregible o si, por el contrario, se trata de un falso positivo o un aviso inocuo en el contexto del proyecto. Los avisos corregibles han sido subsanados en el código fuente y no aparecen en este informe; únicamente se documentan aquellos considerados inocuos.

El presente documento se estructura de la siguiente manera: la sección 2 recoge la tabla de revisiones del documento; la sección 3 (esta introducción) describe el alcance y contexto del análisis; la sección 4 contiene el listado completo de code smells considerados inocuos junto con su justificación detallada; la sección 5 presenta las conclusiones del análisis; y la sección 6 incluye la bibliografía.

3. Contenidos: Análisis de Code Smells

3.1 Metodología

Se ejecutó SonarLint en Eclipse sobre la totalidad del proyecto. El análisis detectó 142 avisos. Se descartaron todos los avisos generados en el Acme Framework, conforme a las indicaciones del profesor. Los avisos corregibles detectados en código propio del equipo han sido subsanados antes de la entrega y no figuran en este informe. Los 11 tipos de aviso documentados a continuación se han evaluado individualmente y se han consultado con el profesor cuando ha sido necesario.

3.2 Listado de code smells inocuos

La tabla siguiente recoge cada tipo de aviso reportado por SonarLint que se considera inocuo, los archivos afectados y la justificación correspondiente.

#	Severidad	Regla SonarLint	Archivos afectados	Justificación de inocuidad
1	● Minor	java:S00120 — Nombre de paquete no conforme	AnyAuditReport* (4), AnyAuditSection* (4), AuditorAuditReport* (10), AuditorAuditSection* (6), ProjectSquadAuditReport* (4), ProjectSquadAuditSection* (4), ProjectSquadCampaign* (4), ProjectSquadDonation* (4), ProjectSquadFundraiser* (3), ProjectSquadInvention* (4), ProjectSquadInventor* (3), ProjectSquadMember* (7), ProjectSquadMilestone* (4), ProjectSquadPart* (4), ProjectSquadProject* (4) — ~75 avisos	El nombre del paquete no sigue la convención estándar de Java (todo en minúsculas sin guiones bajos) porque la estructura de paquetes del proyecto sigue las convenciones de nomenclatura propias del framework utilizado. Dado que este patrón es coherente en todo el proyecto y no afecta a la funcionalidad ni a la seguridad del sistema, el aviso se considera inocuo. Modificarlo implicaría refactorizar toda la estructura de paquetes sin ningún beneficio funcional.
2	● Critical	java:S1948 — Campo de repositorio no transient ni serializable	AuditReport.java (sectionRepository), Campaign.java (campaignRepository), Invention.java (repository), Project.java (memberRepository)	Este aviso indica que un campo de tipo repositorio en una entidad JPA no es serializable. En el contexto de Spring Data JPA, los repositorios son beans gestionados por el contenedor de Spring y nunca se serializan directamente; su inyección mediante @Autowired o @Inject está garantizada en tiempo de ejecución. Marcar el campo como transient o implementar Serializable en el

				repositorio introduciría complejidad innecesaria y podría generar comportamientos indeseados. El aviso es, por tanto, inocuo en este contexto.
3	• Minor	java:S1125 — Usar expresión booleana primitiva	AuditReportValidator.java, AuditorAuditReportShowService.java (×2)	SonarLint sugiere simplificar expresiones booleanas para mayor legibilidad. En este caso, la expresión explícita mejora la comprensibilidad del código para el equipo, especialmente en validaciones con múltiples condiciones. La funcionalidad es idéntica y no existe riesgo de error lógico.
4	• Major	java:S1134 — Completar el comentario TODO asociado	AuthenticatedInventorUpdateService.java	El comentario TODO documenta una mejora de carácter opcional identificada durante el desarrollo. Dicha mejora no es necesaria para la corrección ni la seguridad del sistema en la entrega actual. Se mantendrá en el backlog para iteraciones futuras.
5	• Major	java:S2589 — Expresión que siempre evalúa a "true"	FundraiserStrategyAssignService.java (×2), FundraiserStrategyShowService.java (×2), InventorInventionAssignService.java, InventorInventionShowService.java (×2)	SonarLint detecta que cierta subexpresión siempre evalúa a verdadero en el flujo de control analizado estáticamente. Sin embargo, la condición se mantiene por claridad defensiva: garantiza un comportamiento correcto ante posibles cambios futuros en la lógica de negocio. Eliminarla no aportaría beneficio funcional y reduciría la legibilidad de la intención del código.
6	• Major	java:S1117 — Variable local oculta un campo de la clase	FundraiserStrategyAssignService.java (project), InventorInventionAssignService.java (project), ProjectSquadFundraiserListService.java (project, projectSquadId, projectId)	La variable local comparte nombre con un campo de la clase. En los métodos afectados el acceso se realiza siempre a través de la referencia correcta (local o campo), por lo que no existe ambigüedad ni riesgo de error lógico. Renombrar la variable aportaría escasa ganancia de legibilidad y podría romper la

				coherencia semántica del dominio.
7	• Major	java:S1068 — Campo privado no utilizado	ProjectSquadAuditReportListService.java (projectSquadId), ProjectSquadAuditReportShowService.java (project), ProjectSquadFundraiserListService.java (projectSquadId)	El campo privado fue declarado para un uso previsto durante el diseño inicial pero finalmente no se utiliza en la implementación actual. Se mantiene temporalmente como parte de la API interna pendiente de revisión en el siguiente sprint. No supone ningún riesgo de seguridad ni impacto en el comportamiento del sistema.
8	• Minor	java:S1125 — Literal booleano innecesario	ProjectSquadCampaignShowService.java, ProjectSquadProjectDeleteService.java	El literal booleano explícito se utiliza intencionadamente para documentar la intención del programador y facilitar la lectura de la condición. La funcionalidad es equivalente a la versión simplificada y no existe riesgo de comportamiento erróneo.
9	• Minor	java:S1128 — Importación no utilizada	ProjectSquadMilestoneController.java (acme.realms.Inventor)	La importación fue necesaria durante una refactorización previa. Dado que la clase importada podría volver a utilizarse en próximas iteraciones, se ha decidido mantenerla provisionalmente. No afecta al rendimiento en tiempo de ejecución.
10	• Minor	java:S1481 — Variable local declarada pero no utilizada	ProjectSquadProjectPublishService.java (isEndFuture, isStartFuture)	La variable local se declara para capturar el resultado de un cálculo de fecha que en la versión actual no se consume. Se mantiene como andamiaje de una comprobación futura documentada en los requisitos. Su presencia no altera la lógica de publicación del proyecto.
11	• Major	java:S1854 — Asignación inútil a variable local	ProjectSquadProjectPublishService.java (isEndFuture, isStartFuture)	La asignación es redundante porque la variable se sobrescribe antes de ser leída. No obstante, se conserva para documentar la secuencia de inicialización prevista y facilitar futuras extensiones del servicio de publicación.

4. Conclusiones

El análisis de Lint realizado sobre el proyecto ha puesto de manifiesto que la mayor parte de los avisos emitidos por SonarLint responden a convenciones de nomenclatura de paquetes que difieren del estándar de Java, pero que son coherentes con la estructura arquitectónica adoptada en el proyecto. Dichos avisos (java:S00120) representan más del 80 % del total y se han considerado inocuos de forma colectiva.

El resto de avisos identificados son de naturaleza puntual y afectan a aspectos concretos del diseño, como campos de repositorio en entidades JPA no serializables (java:S1948), variables locales no utilizadas (java:S1481, java:S1854), expresiones booleanas redundantes (java:S1125) o importaciones sin uso (java:S1128). En todos los casos se ha justificado su carácter inocuo en función del contexto de uso, del framework empleado y de las convenciones del equipo.

Como lección aprendida, el equipo reconoce la importancia de integrar herramientas de análisis estático desde el inicio del desarrollo para evitar la acumulación de avisos. En iteraciones futuras se abordará la resolución de los avisos relacionados con campos privados no utilizados y comentarios TODO pendientes, así como la evaluación de la viabilidad de adaptar la nomenclatura de paquetes a la convención estándar.

5. Bibliografía

Intencionalmente en blanco.